

	IES ISIDRA DE GUZMÁN PRUEBA ESCRITA. CONVOCATORIA ORDINARIA Curso 2025-2026 MÓDULO: PROGRAMACIÓN	Calificación
Nombre y apellidos:		PUESTO:
Curso: 1º DESARROLLO APLICACIONES MULTIPLATAFORMA		Fecha: 02-06-26
<u>INSTRUCCIONES</u> <u>LEA DETENIDAMENTE</u> las siguientes instrucciones antes de realizar el examen: Durante la realización del examen, no se podrá hablar ni compartir materiales con otros compañeros. El móvil y el reloj inteligente deben permanecer apagados y fuera del alcance del alumno. El profesor proyectará la hora dentro del aula. Durante la realización del examen: - Se <u>desconectará el cable de red.</u> - Se podrá consultar la ayuda local disponible de la API de Java - Al finalizar el ejercicio, <u>se deberá informar al profesor levantando la mano. Solo si este lo autoriza, se conectará el cable de red y de forma inmediata se subirá al buzón del Aula Virtual.</u> - Si se <u>detecta el empleo de herramientas de inteligencia artificial, copia o cualquier tipo de fraude el examen será calificado con un 0.</u> - Si el profesor lo considera oportuno, podrá citar al alumno para que conteste y razone el ejercicio entregado. - El programa entregado deberá compilar sin errores. En caso contrario, no se podrá evaluar y la nota será un 0. - A lo largo del examen el profesor podrá solicitar copia/s de tu programa. Cuando esto suceda, deberás crear una carpeta con el nombre C_NUMCOPIA_Apellido1 e incluir el fichero .java de ese momento (no importa su estado, si está incompleto o tiene errores). NUMCOPIA será el número de copia comenzando con el número 1 - Por si hubiera un corte de luz, <u>recuerda ir guardando cada poco tiempo el trabajo que estás realizando.</u> Se evalúan los resultados de aprendizaje RA1, RA2, RA3, RA4, RA5, RA6, RA7, RA8, RA9 Duración 4h 30min		

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

REQUISITOS Y ENTREGABLES:

- Revisa tu entrega. No se admitirán entregas una vez finalizado el examen y/o tras haber abandonado el aula. Solo se dispone de un intento.
- Una vez entregada la tarea en el aula virtual, deberás entregar este enunciado y todas las hojas que hayas utilizado
- **Para la evaluación es imprescindible cumplir con los requisitos establecidos anteriormente**

REQUISITOS PARA TODOS LOS EJERCICIOS:

- Crea un proyecto Maven con los requisitos del enunciado
- Incluye los comentarios correspondientes en la cabecera de cada clase, así como comentarios internos para identificar cada requisito dentro del código
- Aplica las buenas prácticas de programación: uso adecuado de comentarios en el código, identificadores camelCase, uso adecuado de las estructuras de control, elección de tipos de datos apropiados, modularización. En caso contrario se penalizará.
- Aplica los conceptos y buenas prácticas de la Programación Orientada a Objetos
- El programa debe codificarse en Java y compilar sin errores. En caso contrario no se podrá evaluar y la nota será 0.
- Penalizaciones:
 - -0,15 por mal uso de identificadores
 - -0,25 por no declarar valores constantes
 - -0,25 por uso inadecuado de tipos de datos
 - -0,5 por no usar comentarios
 - -0,5 mal uso de modificadores de visibilidad
 - -5 puntos por uso de break, return para salir de estructuras de control
 - -10 puntos si no compila
 - -0,5 si no se crean los paquetes y ficheros tal y como se piden
 - -1,5 si no se modulariza correctamente

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Aplicación de gestión de alquileres de vehículos

Contexto

Se desea desarrollar una aplicación de consola en Java para gestionar una empresa de alquiler de vehículos. La empresa dispone de distintos tipos de vehículos, como coches, motos y furgonetas. Todos ellos tendrán datos comunes, pero también información específica según el tipo de vehículo.

La aplicación deberá permitir cargar vehículos desde un fichero de texto, consultar el catálogo, registrar nuevos vehículos, realizar alquileres, devolver vehículos, trabajar con ficheros, insertar vehículos en una base de datos MySQL y aplicar correctamente los principios de programación orientada a objetos.

La aplicación deberá estar correctamente estructurada en clases, aplicando modularización, encapsulación, herencia, polimorfismo, tratamiento de errores y buenas prácticas de programación.

Desarrollar una aplicación de consola en Java que permita las **siguientes operaciones**:

===== GESTIÓN DE ALQUILERES DE VEHÍCULOS =====

1. Cargar vehículos desde fichero
2. Alta de vehículo
3. Mostrar todos los vehículos
4. Buscar vehículo por matrícula/código
5. Realizar alquiler
6. Devolver vehículo
7. Mostrar alquileres
8. Exportar vehículos a fichero de texto
9. Guardar alquileres en fichero binario
10. Cargar alquileres desde fichero binario
11. Insertar vehículos en base de datos
0. Salir

La aplicación deberá representar mediante clases los principales elementos que intervienen en la gestión de alquileres de una empresa de vehículos.

La empresa trabaja con distintos tipos de vehículos alquilables. Todos los vehículos comparten una serie de características comunes, como su identificación, marca, modelo, año de fabricación, precio base por día y estado de disponibilidad. Sin embargo, cada tipo de vehículo también tiene información específica que lo diferencia del resto.

Por este motivo, el diseño deberá incluir una estructura de herencia que permita representar los elementos comunes de todos los vehículos y, al mismo tiempo, especializar el comportamiento de cada tipo concreto.

La aplicación deberá contemplar tres tipos distintos de vehículos:

Tipo de vehículo	Datos específicos
Coche	Número de puertas y tipo de combustible
Moto	Cilindrada y si tiene baúl
Furgoneta	Capacidad de carga en kg y volumen en m ³

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Requisito sobre los códigos de los vehículos

1. Cada vehículo deberá tener un código identificativo único. El código deberá permitir distinguir el tipo de vehículo, el año asociado y un número correlativo.
2. El formato obligatorio será: TIPO-AAAA-NNN

Donde:

Parte	Significado
TIPO	Prefijo correspondiente al tipo de vehículo
AAAA	Año con cuatro cifras
NNN	Número correlativo de tres cifras

Los prefijos obligatorios serán:

Tipo de vehículo	Prefijo
Coche	COC
Moto	MOT
Furgoneta	FUR

Ejemplos válidos:

- COC-2020-001
- MOT-2023-002
- FUR-2021-003

Además, el prefijo del código deberá coincidir con el tipo de vehículo seleccionado. No se podrá registrar una moto con un código que empiece por COC o FUR.

Comportamiento específico de cada tipo de vehículo

Cada tipo de vehículo tendrá unas condiciones de alquiler diferentes. La aplicación deberá permitir calcular automáticamente:

- El número máximo de días que puede alquilarse cada vehículo.
- El coste diario del alquiler.
- La penalización económica por cada día de retraso en la devolución.

Estos cálculos deberán depender del tipo concreto de vehículo. Por tanto, el diseño deberá incluir algún mecanismo que obligue a cada clase específica a definir su propio comportamiento.

Las condiciones mínimas serán las siguientes:

Tipo de vehículo	Días máximos de alquiler	Coste diario	Penalización por día de retraso
Coche	15 días	45,00 €	20,00 €
Moto	7 días	25,00 €	12,00 €
Furgoneta	10 días	65,00 €	30,00 €

Requisitos sobre disponibilidad de vehículos

- Cada vehículo deberá conocer si está disponible o no.
- Cuando se realice un alquiler correctamente:
 - El vehículo deberá pasar a estar no disponible.
 - No se deberá permitir alquilar de nuevo ese mismo vehículo mientras esté alquilado.

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

- Cuando se registre la devolución:
 - El vehículo deberá volver a estar disponible.
 - El alquiler deberá quedar marcado como devuelto.
- El alumnado deberá decidir qué métodos son necesarios para cambiar y consultar el estado de disponibilidad de un vehículo.

Requisitos sobre cómo almacenar la información

La aplicación deberá almacenar en memoria:

- El catálogo de vehículos de la empresa.
- El conjunto de alquileres realizados.

El alumnado deberá seleccionar las colecciones más adecuadas para cada caso y justificar su elección mediante comentarios internos en el código.

Se valorará especialmente que:

- Los vehículos puedan localizarse eficientemente por su código.
- Se mantenga un recorrido claro y ordenado del catálogo.
- Los alquileres puedan almacenarse y mostrarse de forma secuencial.

Requisito funcional opción 1: Cargar vehículos desde fichero

Al iniciar la aplicación, el catálogo de vehículos podrá estar vacío. No obstante, deberá existir una opción del menú que permita cargar vehículos desde un fichero de texto: `files/vehiculos_iniciales.txt`

Cada línea del fichero representará un vehículo y tendrá un formato separado por punto y coma.

Ejemplo:

`COC-2020-001;Coche;Toyota;Corolla;2020;true;5;Gasolina`

`MOT-2023-001;Moto;Yamaha;MT-07;2023;true;689;true`

`FUR-2021-001;Furgoneta;Ford;Transit;2021;true;1200;11.5`

La aplicación deberá leer el fichero, interpretar cada línea y crear el objeto correspondiente según el tipo de vehículo indicado.

Se deberá validar que:

- El fichero existe.
- Cada línea tiene el número de campos esperado.
- El código cumple el patrón establecido.
- El prefijo del código coincide con el tipo de vehículo.
- El vehículo no está duplicado.
- Los datos numéricos tienen formato correcto.
- El tipo de vehículo es reconocido por la aplicación.

Si una línea contiene errores, la aplicación no deberá finalizar. Se mostrará un aviso indicando la línea incorrecta y se continuará con la siguiente.

Requisito funcional opción 2: Alta de vehículo

La aplicación deberá permitir registrar manualmente un nuevo vehículo en el catálogo.

Al seleccionar esta opción, el programa mostrará un submenú:

1. Coche
2. Moto
3. Furgoneta

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Una vez seleccionado el tipo de vehículo, la aplicación solicitará los datos comunes:

- Código identificativo.
- Marca.
- Modelo.
- Año de fabricación.

Todo vehículo dado de alta manualmente se incorporará inicialmente como disponible.

Además, según el tipo de vehículo elegido, se pedirán los datos específicos:

Coche

- Número de puertas.
- Tipo de combustible.

Moto

- Cilindrada.
- Si tiene baúl.

Furgoneta

- Capacidad de carga en kg.
- Volumen en m³.
- Antes de incorporar el vehículo al catálogo, la aplicación deberá validar, como mínimo, que:
- El código no esté vacío.
- El código cumpla el patrón mediante una expresión regular.
- El prefijo del código coincida con el tipo de vehículo seleccionado.
- No exista ya otro vehículo con el mismo código.
- La marca no esté vacía.
- El modelo no esté vacío.
- El año de fabricación sea un valor numérico válido.
- Los datos específicos sean coherentes.
- Los valores numéricos específicos sean mayores que cero.

Si los datos son correctos, la aplicación creará un objeto del tipo correspondiente y lo almacenará en la colección utilizada para gestionar el catálogo.

Mensaje orientativo:

Vehículo dado de alta correctamente.

Código: COC-2020-001

Marca: Toyota

Modelo: Corolla

Tipo: Coche

Si se detecta algún error, la aplicación deberá mostrar un mensaje claro y controlado, sin finalizar inesperadamente. El mensaje de error se mostrará en color rojo.

Requisito funcional opción 3: Mostrar todos los vehículos

La aplicación deberá permitir mostrar por pantalla todos los vehículos almacenados actualmente en el catálogo.

Al seleccionar esta opción, el programa recorrerá con un **iterador** la colección utilizada para guardar los vehículos y mostrará la información de cada uno de ellos de forma clara y ordenada.

La salida deberá incluir, como mínimo:

- Código.
- Tipo de vehículo.
- Marca.
- Modelo.
- Año de fabricación.

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

- Estado de disponibilidad.
- Coste diario.
- Datos específicos de cada tipo de vehículo.

La información se muestre en formato tabla, con alineación en columnas y uso de colores. Por ejemplo, si el vehículo no está disponible, la fila podrá mostrarse en color rojo.

Requisito funcional opción 4: Buscar vehículo por código

La aplicación deberá permitir buscar un vehículo concreto del catálogo a partir de su código identificativo.

El código introducido deberá cumplir el formato obligatorio: TIPO-AAAA-NNN

Antes de realizar la búsqueda, la aplicación deberá validar que:

- El código no esté vacío.
- El código cumple el patrón establecido mediante una expresión regular.
- El prefijo corresponde a uno de los tipos permitidos.

Si el código no tiene un formato válido, se mostrará un mensaje de error controlado:

El código introducido no tiene un formato válido.

Recuerde que debe seguir el formato COC-AAAA-NNN, MOT-AAAA-NNN o FUR-AAAA-NNN.

Si el vehículo existe, se mostrará toda su información:

- Código.
- Tipo.
- Marca.
- Modelo.
- Año de fabricación.
- Estado de disponibilidad.
- Datos específicos.
- Días máximos de alquiler.
- Coste diario.
- Penalización por día de retraso.

Ejemplo orientativo:

===== VEHÍCULO ENCONTRADO =====

Código: COC-2020-001

Tipo: Coche

Marca: Toyota

Modelo: Corolla

Año de fabricación: 2020

Disponible: Sí

Datos específicos: 5 puertas, Gasolina

Días máximos de alquiler: 15

Coste diario: 45,00 €

Penalización por día de retraso: 20,00 €

Si no existe ningún vehículo con ese código, se mostrará:

No existe ningún vehículo registrado con el código COC-2020-001.

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Requisito funcional opción 5: Realizar alquiler

La aplicación deberá permitir registrar el alquiler de un vehículo a un cliente.

Al seleccionar esta opción, el programa solicitará:

- Código del vehículo.
- Nombre del cliente.

Antes de crear el alquiler, la aplicación deberá comprobar:

- El código del vehículo no puede estar vacío.
- El código debe cumplir el patrón obligatorio mediante expresión regular.
- El vehículo debe existir en el catálogo.
- El vehículo debe estar disponible.
- El nombre del cliente no puede estar vacío.

Si el vehículo no existe, se mostrará:

No existe ningún vehículo registrado con el código indicado.

Si el vehículo existe pero no está disponible, se mostrará:

El vehículo seleccionado no está disponible actualmente para alquiler.

Si todos los datos son correctos, la aplicación deberá crear un nuevo alquiler asociado al vehículo seleccionado y al cliente.

El alquiler deberá incluir, como mínimo:

- Identificador único de alquiler.
- Nombre del cliente.
- Vehículo alquilado.
- Fecha de alquiler, que será la fecha actual del sistema.
- Fecha prevista de devolución.
- Estado: PENDIENTE o DEVUELTO.

El identificador podrá generarse automáticamente con el patrón: ALQ001,ALQ002,ALQ003...

La fecha prevista de devolución se calculará automáticamente según el tipo de vehículo:

Tipo de vehículo	Días máximos de alquiler
Coche	15 días
Moto	7 días
Furgoneta	10 días

Una vez creado el alquiler:

- El vehículo deberá marcarse como no disponible.
- El alquiler deberá almacenarse en la colección utilizada para gestionar los alquileres.
- Se deberá mostrar un mensaje de confirmación.

Ejemplo:

Alquiler creado correctamente.

ID alquiler: ALQ001

Cliente: Laura Martínez

Vehículo: COC-2020-001 - Toyota Corolla

Fecha de alquiler: 07/05/2026

Fecha prevista de devolución: 22/05/2026

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Estado: PENDIENTE

Requisito funcional opción 6: Devolver vehículo

La aplicación deberá permitir registrar la devolución de un vehículo alquilado previamente.

Al seleccionar esta opción, el programa solicitará:

Introduce el ID del alquiler: ALQ001

La aplicación deberá buscar ese alquiler en la colección utilizada para almacenar los alquileres realizados.

Antes de registrar la devolución, el programa deberá comprobar:

- Que el identificador del alquiler no esté vacío.
- Que existe un alquiler con ese identificador.
- Que el alquiler todavía está pendiente de devolución.
- Que el vehículo asociado al alquiler existe y puede volver a marcarse como disponible.

Si el alquiler no existe, se mostrará:

No existe ningún alquiler registrado con el identificador indicado.

Si el alquiler ya fue devuelto anteriormente, se mostrará:

Este alquiler ya fue devuelto anteriormente.

Si el alquiler existe y está pendiente, la aplicación deberá registrar la devolución.

Para facilitar la corrección, se permitirá introducir manualmente la fecha real de devolución:

Introduce la fecha real de devolución (dd/MM/yyyy):

La aplicación deberá comparar la fecha real de devolución con la fecha prevista de devolución.

Si la fecha real es posterior a la fecha prevista, existirá retraso. En ese caso, se calcularán:

- Los días de retraso.
- La penalización económica según el tipo de vehículo.
- El importe total del alquiler.

El cálculo de la penalización deberá realizarse utilizando el comportamiento propio del vehículo alquilado mediante polimorfismo.

Cuando la devolución sea correcta, la aplicación deberá:

- Registrar la fecha real de devolución.
- Marcar el alquiler como devuelto.
- Cambiar el estado a DEVUELTO.
- Marcar el vehículo asociado como disponible.
- Calcular los días de retraso, si los hay.
- Calcular la penalización correspondiente.
- Mostrar un resumen de la operación.

Ejemplo de devolución sin retraso:

Vehículo devuelto correctamente.

ID alquiler: ALQ001

Cliente: Laura Martínez

Vehículo: COC-2020-001 - Toyota Corolla

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Fecha prevista de devolución: 22/05/2026
 Fecha real de devolución: 21/05/2026
 Estado del alquiler: DEVUELTO
 Penalización: 0,00 €

Ejemplo con retraso:

Vehículo devuelto correctamente.

ID alquiler: ALQ002
 Cliente: Carlos Gómez
 Vehículo: MOT-2023-001 - Yamaha MT-07
 Fecha prevista de devolución: 14/05/2026
 Fecha real de devolución: 17/05/2026
 Estado del alquiler: DEVUELTO
 Días de retraso: 3
 Penalización: 36,00 €

Requisito funcional opción 7: Mostrar alquileres

La aplicación deberá permitir mostrar por pantalla todos los alquileres registrados durante la ejecución del programa. Cada alquiler deberá mostrar, como mínimo:

- Identificador del alquiler.
- Nombre del cliente.
- Código del vehículo.
- Marca y modelo.
- Tipo de vehículo.
- Fecha de alquiler.
- Fecha prevista de devolución.
- Fecha real de devolución, si ya ha sido devuelto.
- Estado: PENDIENTE o DEVUELTO.
- Días de retraso, si procede.
- Penalización, si procede.

Ejemplo:

ID alquiler: ALQ002
 Cliente: Carlos Gómez
 Vehículo: MOT-2023-001 - Yamaha MT-07
 Tipo de vehículo: Moto
 Fecha de alquiler: 07/05/2026
 Fecha prevista de devolución: 14/05/2026
 Fecha real de devolución: 17/05/2026
 Estado: DEVUELTO
 Días de retraso: 3
 Penalización: 36,00 €

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Si el alquiler todavía está pendiente:

Fecha real de devolución: -
Estado: PENDIENTE
Penalización: No calculada

Si no existen alquileres registrados, la aplicación deberá mostrar:

No hay alquileres registrados.

Requisito funcional opción 8: Exportar vehículos a fichero de texto

La aplicación deberá permitir exportar a un fichero de texto todos los vehículos almacenados actualmente en el catálogo. El fichero se llamará: files/vehiculos.txt

Cada línea representará un vehículo. Los datos deberán escribirse separados por punto y coma. Ejemplo:

COC-2020-001;Coche;Toyota;Corolla;2020;true;5;Gasolina

MOT-2023-001;Moto;Yamaha;MT-07;2023;false;689;true

FUR-2021-001;Furgoneta;Ford;Transit;2021;true;1200;11.5

Cada línea deberá incluir, como mínimo:

- Código.
- Tipo.
- Marca.
- Modelo.
- Año de fabricación.
- Estado de disponibilidad.
- Datos específicos del tipo de vehículo.

Si no existen vehículos registrados, se mostrará:

No hay vehículos para exportar.

Si la exportación se realiza correctamente:

Vehículos exportados correctamente.
Fichero generado: files/vehiculos.txt
Vehículos exportados: 8

La aplicación deberá controlar posibles errores durante la escritura del fichero.

Requisito funcional opción 9: Guardar alquileres en fichero binario

La aplicación deberá permitir guardar en un fichero binario todos los alquileres registrados durante la ejecución del programa. El fichero se llamará: files/alquileres.dat

La operación se realizará mediante serialización de objetos.

Para poder realizar esta operación, deberán implementar Serializable las clases necesarias, especialmente:

- Alquiler
- Vehiculo
- Coche
- Moto
- Furgoneta

Si no existen alquileres registrados, se mostrará:

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

No hay alquileres registrados para guardar.

Si la operación se realiza correctamente:

Alquileres guardados correctamente.

Fichero generado: files/alquileres.dat

Alquileres guardados: 5

Requisito funcional opción 10: Cargar alquileres desde fichero binario

La aplicación deberá permitir cargar desde un fichero binario los alquileres guardados previamente mediante serialización. El fichero que deberá leer será: files/alquileres.dat

La aplicación deberá comprobar si el fichero existe antes de intentar leerlo.

Si el fichero no existe, se mostrará:

No existe ningún fichero de alquileres guardado.

Debe guardar alquileres previamente antes de poder cargarlos.

Si el fichero existe y se carga correctamente, los alquileres recuperados deberán incorporarse a la colección utilizada por la aplicación para gestionar los alquileres.

Mensaje orientativo:

Alquileres cargados correctamente.

Fichero leído: files/alquileres.dat

Alquileres recuperados: 5

Requisito funcional opción 11: Insertar vehículos en base de datos

La aplicación deberá permitir insertar en una base de datos MySQL los vehículos almacenados actualmente en el catálogo. Al seleccionar la opción 11, el programa deberá recorrer la colección utilizada para guardar los vehículos y registrar cada uno de ellos en la tabla correspondiente de la base de datos.

La inserción deberá realizarse utilizando:

- Conexión JDBC.
- Clase DBConnection.
- Clase DAO, por ejemplo VehiculoDAO.
- PreparedStatement.
- Control de duplicados.

Si un vehículo ya existe en la base de datos, la aplicación no deberá finalizar inesperadamente. Se mostrará un aviso y continuará con el siguiente vehículo.

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

REQUISITOS TÉCNICOS OBLIGATORIOS

La aplicación deberá cumplir obligatoriamente los siguientes requisitos:

Organización del proyecto

- El proyecto debe estructurarse según funcionalidades. Los ficheros deberán organizarse en paquetes. Ej:

```
src/main/java/examen
├── controlador/
│   └── AppController.java
├── dao/
│   └── VehiculoDAO.java
├── excepciones/
│   ├── DatoNoValidoException.java
│   ├── VehiculoNoDisponibleException.java
│   └── ...
├── modelo/
│   ├── Vehiculo.java
│   └── ...
├── utils/
│   ├── InputReader.java
│   ├── Ansi.java
│   └── DBConnection.java
├── vista/
│   └── ConsoleView.java
└── AppRentacar.java
```

Se podrán añadir otras capas o paquetes auxiliares si se considera necesario.

Estilo de programación

- La aplicación deberá tener una clase principal con el método main, pero este método deberá contener el menor código posible. La lógica de la aplicación deberá estar repartida en clases y métodos auxiliares.
- Los identificadores deben seguir la convención **camelCase** cuando corresponda.
- Los nombres de clases y variables deben ser significativos.
- El código debe estar correctamente indentado y ordenado. Separación adecuada de responsabilidades.
- Se deben incluir:
 - **comentarios internos**
 - **comentarios Javadoc** en clases y métodos importantes. Al menos en la clase RecursoBibliografico

Utilidades obligatorias

Dentro del paquete utils deberán existir al menos los siguientes ficheros:

- InputReader.java: Para centralizar la lectura y validación de datos introducidos por teclado.
- Ansi.java: Para definir constantes de colores ANSI y mejorar la experiencia visual del usuario en consola. Dispones de este fichero en el directorio de los recursos compartidos en el directorio home del usuario

Nombre y apellidos:	Puesto:
Curso: 1º Desarrollo Aplicaciones Multiplataforma	

Requisitos sobre excepciones

- La aplicación deberá controlar errores de forma adecuada, evitando que el programa finalice de manera inesperada.
- El alumnado deberá crear y utilizar al menos una excepción personalizada relacionada con alguna situación propia del problema.
- Por ejemplo:
 - Código de vehículo no válido.
 - Vehículo inexistente.
 - Vehículo no disponible.
 - Dato obligatorio vacío.
 - Valor numérico no válido.
 - Alquiler inexistente.
 - Alquiler ya devuelto.
- La excepción o excepciones deberán utilizarse en operaciones relevantes del programa, no solo declararse sin uso real.